

Repositorios Específicos

Objetivos

- Agregar métodos específicos a repositorios
- Mantener queries complejas organizadas
- Usar eager loading desde el repositorio
- Crear interfaces claras para cada entidad

Extendiendo BaseRepository

BaseRepository proporciona operaciones genéricas. Los repositorios específicos agregan **métodos particulares** para cada entidad.

Patrón de Extensión

```
from sqlalchemy import select
from sqlalchemy.orm import Session, selectinload

from models import Author
from repositories.base import BaseRepository

class AuthorRepository(BaseRepository[Author]):
    """Repositorio con métodos específicos para Author"""

    def __init__(self, db: Session):
        super().__init__(db, Author)

    # --- Métodos específicos ---

    def get_by_email(self, email: str) -> Author | None:
        """Busca autor por email único"""
        return self.first(email=email)

    def get_with_posts(self, author_id: int) -> Author | None:
        """Obtiene autor con sus posts cargados (eager loading)"""
        stmt = (
            select(Author)
            .options(selectinload(Author.posts))
            .where(Author.id == author_id)
        )
        return self.db.execute(stmt).scalar_one_or_none()

    def get_active_authors(self) -> list[Author]:
        """Obtiene autores con al menos 1 post"""
        stmt = (
            select(Author)
            .join(Author.posts)
            .distinct()
        )
        return list(self.db.execute(stmt).scalars().all())

    def search_by_name(self, query: str) -> list[Author]:
```

```
"""Busca autores por nombre (case-insensitive)"""
stmt = select(Author).where(
    Author.name.ilike(f"%{query}%")
)
return list(self.db.execute(stmt).scalars().all())
```

Ejemplos por Tipo de Entidad

TaskRepository (Proyecto de la semana)

```
from datetime import date
from sqlalchemy import select, and_
from sqlalchemy.orm import Session, joinedload

from models import Task, TaskStatus
from repositories.base import BaseRepository

class TaskRepository(BaseRepository[Task]):
    """Repositorio para operaciones de Task"""

    def __init__(self, db: Session):
        super().__init__(db, Task)

    def get_with_user(self, task_id: int) -> Task | None:
        """Obtiene task con usuario cargado"""
        stmt = (
            select(Task)
            .options(joinedload(Task.user))
            .where(Task.id == task_id)
        )
        return self.db.execute(stmt).scalar_one_or_none()

    def get_by_user(
        self,
        user_id: int,
        status: TaskStatus | None = None
    ) -> list[Task]:
        """Obtiene tasks de un usuario, opcionalmente filtradas por status"""
        stmt = select(Task).where(Task.user_id == user_id)

        if status:
            stmt = stmt.where(Task.status == status)

        stmt = stmt.order_by(Task.due_date.asc().nulls_last())
        return list(self.db.execute(stmt).scalars().all())

    def get_pending(self, user_id: int | None = None) -> list[Task]:
        """Obtiene tasks pendientes (no completadas)"""
        stmt = select(Task).where(Task.status != TaskStatus.DONE)

        if user_id:
            stmt = stmt.where(Task.user_id == user_id)
```

```

        return list(self.db.execute(stmt).scalars().all())

def get_overdue(self) -> list[Task]:
    """Obtiene tasks vencidas"""
    stmt = select(Task).where(
        and_(
            Task.due_date < date.today(),
            Task.status != TaskStatus.DONE
        )
    )
    return list(self.db.execute(stmt).scalars().all())

def get_due_today(self) -> list[Task]:
    """Obtiene tasks que vencen hoy"""
    stmt = select(Task).where(Task.due_date == date.today())
    return list(self.db.execute(stmt).scalars().all())

def count_by_status(self, user_id: int) -> dict[TaskStatus, int]:
    """Cuenta tasks por status para un usuario"""
    from sqlalchemy import func

    stmt = (
        select(Task.status, func.count(Task.id))
        .where(Task.user_id == user_id)
        .group_by(Task.status)
    )
    results = self.db.execute(stmt).all()
    return {status: count for status, count in results}

```

UserRepository

```

from sqlalchemy import select
from sqlalchemy.orm import Session, selectinload

from models import User
from repositories.base import BaseRepository

class UserRepository(BaseRepository[User]):
    """Repositorio para operaciones de User"""

    def __init__(self, db: Session):
        super().__init__(db, User)

    def get_by_email(self, email: str) -> User | None:
        """Busca usuario por email"""
        return self.first(email=email)

    def get_by_username(self, username: str) -> User | None:
        """Busca usuario por username"""
        return self.first(username=username)

    def get_with_tasks(self, user_id: int) -> User | None:
        """Obtiene usuario con todas sus tasks"""
        stmt = (

```

```

        select(User)
        .options(selectinload(User.tasks))
        .where(User.id == user_id)
    )
    return self.db.execute(stmt).scalar_one_or_none()

def get_active_users(self) -> list[User]:
    """Obtiene usuarios activos"""
    return self.filter_by(is_active=True)

def email_exists(self, email: str) -> bool:
    """Verifica si el email ya está registrado"""
    return self.get_by_email(email) is not None

```

🎯 Principios para Métodos Específicos

1. Nombres Descriptivos

```

# ✅ BIEN - nombres claros
def get_pending_by_user(self, user_id: int) -> list[Task]: ...
def get_overdue(self) -> list[Task]: ...
def count_by_status(self, user_id: int) -> dict: ...

# ❌ MAL - nombres vagos
def get_tasks(self, user_id: int, status: str) -> list[Task]: ...
def fetch(self) -> list[Task]: ...

```

2. Un Método, Una Responsabilidad

```

# ✅ BIEN - métodos separados
def get_pending(self) -> list[Task]: ...
def get_completed(self) -> list[Task]: ...
def get_overdue(self) -> list[Task]: ...

# ❌ MAL - método con demasiadas opciones
def get_tasks(
    self,
    pending: bool = False,
    completed: bool = False,
    overdue: bool = False,
    user_id: int | None = None,
    ...
) -> list[Task]: ...

```

3. Eager Loading en Repository

```

# ✅ Métodos específicos para cargar relaciones
def get_with_posts(self, author_id: int) -> Author | None:
    """Carga autor CON posts"""
    stmt = select(Author).options(selectinload(Author.posts))...

def get_by_id(self, author_id: int) -> Author | None:

```

```
"""Carga autor SIN posts (lazy)"""
return self.db.get(Author, author_id)
```

4. Retornos Tipados

```
# ✅ BIEN - tipos explícitos
def get_by_email(self, email: str) -> User | None: ...
def get_pending(self) -> list[Task]: ...
def count_by_status(self) -> dict[TaskStatus, int]: ...

# ❌ MAL - sin tipos
def get_by_email(self, email): ...
def get_pending(self): ...
```



Estructura Recomendada

```
repositories/
├─ __init__.py
├─ base.py           # BaseRepository[T]
├─ user_repository.py # UserRepository
├─ task_repository.py # TaskRepository
└─ tag_repository.py # TagRepository
```

`__init__.py`

```
from repositories.base import BaseRepository
from repositories.user_repository import UserRepository
from repositories.task_repository import TaskRepository

__all__ = [
    "BaseRepository",
    "UserRepository",
    "TaskRepository",
]
```



Queries Complejas

Para queries muy complejas, mantén el código organizado:

```
class TaskRepository(BaseRepository[Task]):

    def get_dashboard_summary(self, user_id: int) -> dict:
        """
        Obtiene resumen para dashboard del usuario.

        Returns:
            {
                "total": int,
                "pending": int,
                "completed": int,
                "overdue": int,
            }
        """
```

```

        "due_today": int
    }
"""
from sqlalchemy import func, case

stmt = (
    select(
        func.count(Task.id).label("total"),
        func.sum(case((Task.status == TaskStatus.TODO, 1), else_=0)).label("pending"),
        func.sum(case((Task.status == TaskStatus.DONE, 1), else_=0)).label("completed"),
        func.sum(case(
            (and_(Task.due_date < date.today(), Task.status != TaskStatus.DONE), 1),
            else_=0
        )).label("overdue"),
        func.sum(case((Task.due_date == date.today(), 1), else_=0)).label("due_today"),
    )
    .where(Task.user_id == user_id)
)

result = self.db.execute(stmt).one()

return {
    "total": result.total or 0,
    "pending": result.pending or 0,
    "completed": result.completed or 0,
    "overdue": result.overdue or 0,
    "due_today": result.due_today or 0,
}

```

✓ Checklist

- ☐ Sé cuándo agregar métodos específicos vs usar BaseRepository
- ☐ Uso nombres descriptivos para métodos
- ☐ Implemento eager loading en métodos que lo necesitan
- ☐ Mantengo los repositorios enfocados en acceso a datos

Siguiente

Aprenderemos el patrón **Unit of Work** para manejar transacciones de forma coordinada.

→ [04-unit-of-work.md](#)